

Relatório de Integração — MOM (RabbitMQ)

Sprint 2 – Middleware Orientado a Mensagens | 25/05/2026

Disciplina	Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas
Professores	Cleiton Silva Tavares e Cristiano de Macedo Neto
Entrega	Sprint 2 – 25/05/2026

1. Escolha da Ferramenta: RabbitMQ

O RabbitMQ foi escolhido como Middleware Orientado a Mensagens (MOM) por ser a ferramenta citada como exemplo no próprio enunciado do projeto e por ser amplamente adotada na indústria para comunicação assíncrona entre serviços. Suas principais características que motivaram a escolha são:

- **Protocolo AMQP 0-9-1:** protocolo aberto e robusto, com garantia de entrega e suporte a filas duráveis.
- **Painel de gerenciamento (Management UI):** interface web na porta 15672 que permite visualizar filas, mensagens e consumidores em tempo real — essencial para demonstrar o funcionamento durante a apresentação.
- **Mensagens persistentes:** com `delivery_mode=2` as mensagens sobrevivem a reinicializações do broker.
- **Instalação local simples:** disponível para Windows, macOS e Linux com poucos comandos.
- **Biblioteca pika:** cliente Python oficial, estável e bem documentado.

2. Padrão Utilizado: Direct Exchange (Publish/Subscribe)

O padrão adotado é o **Direct Exchange** do RabbitMQ, que é o exchange padrão (string vazia). Neste padrão, o producer publica mensagens com uma `routing_key` igual ao nome da fila de destino, e o broker entrega diretamente àquela fila. Os consumers se inscrevem em filas específicas e processam as mensagens de forma assíncrona, sem qualquer chamada REST direta entre producer e consumer.

Este padrão foi escolhido por sua simplicidade e aderência ao domínio: cada tipo de evento tem sua própria fila nomeada (`order.created`, `order.status_updated`, `product.sold_out`), o que torna o fluxo de eventos explícito, rastreável e fácil de demonstrar.

Fluxo de publicação:

```
1. Comprador faz POST /api/v1/orders → 2. Service cria o pedido no banco → 3. Service chama publish_order_created(order) → 4. Publisher abre conexão AMQP → 5. Publica na fila 'order.created' → 6. Consumer recebe e processa de forma assíncrona
```

3. Documentação dos Eventos

Evento	Producer	Consumer	Disparado quando	Fila
order.created	OrderService (backend Flask)	App Vendedor (Flutter)	Comprador realiza um novo pedido com sucesso	order.created

order.status_updated	OrderService (backend Flask)	App Comprador (Flutter)	Vendedor atualiza o status do pedido (confirmed, shipped, delivered, cancelled)	order.status_updated
product.sold_out	OrderService (backend Flask)	App Vendedor (Flutter)	Estoque do produto chega a zero após um pedido	product.sold_out

4. Payloads JSON de Exemplo

Evento: order.created

```
{ "event": "order.created", "timestamp": "2026-05-25T14:30:00+00:00", "data": {
  "order_id": 1, "buyer_id": 2, "buyer_name": "Carlos Comprador", "product_id": 1,
  "product_title": "Notebook Dell XPS 13", "quantity": 2, "total_price": 9000.0, "status":
  "pending" } }
```

Evento: order.status_updated

```
{ "event": "order.status_updated", "timestamp": "2026-05-25T14:35:00+00:00", "data": {
  "order_id": 1, "buyer_id": 2, "product_id": 1, "product_title": "Notebook Dell XPS 13",
  "previous_status": "pending", "new_status": "confirmed", "total_price": 9000.0 } }
```

Evento: product.sold_out

```
{ "event": "product.sold_out", "timestamp": "2026-05-25T14:30:05+00:00", "data": {
  "product_id": 1, "product_title": "Notebook Dell XPS 13", "seller_id": 1 } }
```

5. Decisões de Design

Falha silenciosa no publisher: O publisher captura exceções de conexão com o RabbitMQ e registra em log, mas não propaga o erro para o controller. Isso garante que uma falha momentânea no broker não derrube o endpoint REST. A operação no banco já foi confirmada; o evento pode ser republicado em uma futura estratégia de retry.

Mensagens persistentes (delivery_mode=2): Toda mensagem publicada usa delivery_mode=2, que instrui o RabbitMQ a gravar a mensagem em disco antes de confirmar. Combinado com filas durable=True, isso garante que nenhuma mensagem seja perdida em caso de reinicialização do broker.

Filas nomeadas por evento: Cada evento tem sua própria fila (order.created, order.status_updated, product.sold_out) em vez de uma fila genérica. Isso permite que consumers diferentes se inscrevam apenas nos eventos de seu interesse, seguindo o princípio de segregação de responsabilidades.

Consumer com reconexão automática: O consumer implementa um loop de reconexão com espera de 5 segundos entre tentativas. Isso o torna resiliente a reinicializações do broker durante desenvolvimento e demonstrações.

ACK explícito por mensagem: O consumer usa basic_ack após processar cada mensagem com sucesso e basic_nack em caso de erro. O RabbitMQ só remove a mensagem da fila após o ACK, garantindo que mensagens com erro sejam reprocessadas (requeue=True) ou descartadas (requeue=False para JSON inválido).

6. Desafios Encontrados

- **Importação circular:** integrar o publisher ao service sem criar dependências circulares entre módulos. Solução: criar a pasta messaging/ com publisher.py independente, importado pelo service

após a inicialização do app.

- **RabbitMQ não disponível:** garantir que o backend não quebre se o RabbitMQ não estiver rodando. Solução: captura de `AMQPConnectionError` no publisher com log de erro sem relançamento da exceção.

- **Contexto Flask no consumer:** o consumer roda fora do app Flask, sem acesso ao banco. Solução: consumer atua apenas como processador de notificações (log + push), sem acesso direto ao banco de dados.